Dimensionality Reduction

Group 20

March 2024

1 Introduction

1.1 *What is Data?*

Any data science literature must first answer the question: What precisely is *data*? While we can recognise data when encountered, providing it with a definition can be a bit challenging. For the purpose of this paper, we will adopt the following definition from *Dimensionality Reduction in Data Science* [5]:

Definition 1. *Data is an objective recording of one or several event(s) in the real world that is accessible at later times for perusal and analysis by at least one person [5].*

Data manifests in diverse forms such as numbers, DNA, images, text etc. These manifestations are regarded as *raw* data and require transformation and processing to facilitate the type of analysis required to address a problem in data science, a topic that will be discussed in the upcoming sections.

1.1.1 Big Data

Despite computational advancements and capabilities in today's time, existing computational resources do not have the processing capacity for the extraordinary amount of data produced across all fields. With these computational resources, extracting valuable information from big data could be difficult because of the overwhelming expansion of data collection. A common concern arises in selecting an appropriate subset of the data to build more suitable models. Subsequently, we will explore the significance of data reduction, particularly in the context of big data, as a crucial step aimed at transforming large datasets into meaningful information, which constitutes the primary objective of data science.

1.1.2 Dataset

Let us consider the measurement of a specific quantity, such as personal health information or images from particular scenes. This quantity can be multidimensional, essentially a collection of values, allowing us to perceive each quantity as a point within a multidimensional space. Suppose the dimensionality of this space is denoted as d, indicating that each quantity is represented by a d-dimensional vector within $\mathbb{R}^d$. The group of d values associated with the quantity are called *features* of the quantity. It is possible to have multiple measurements for a single quantity, each represented as a d-dimensional point. Consequently, this collection of d-dimensional points is referred to as a *dataset*.

1.2 Feature Extraction

A dataset can undergo compression while retaining most of the important information. Thus, we can apply processing to the features to achieve compression, viewed as a pre-processing stage, where the data's dimensionality is reduced. This procedure is called *Feature engineering*, involving two main approaches: feature selection and feature extraction. Feature extraction, also called dimensionality reduction, involves converting a d-dimensional data vector into a p-dimensional one. In this transformation, the p new features are distinct from the original ones, leading to an improved data representation. This report will focus on the feature extraction approach.

The references used for this section:

Benyamin Ghojogh et al. Elements of Dimensionality Reduction and Manifold Learning. Springer Nature, 2023. isbn: 9783031106026.[6]

Max Garzon et al. Dimensionality Reduction in Data Science. Springer Cham, 28 July 2022. isbn: 978303105373. doi: https://doi.org/10.1007/978-3-031-05371-9.[5]

Sylvain Lespinats, Benoit Colange, and Denys Dutykh. Nonlinear Dimensionality Reduction Techniques. Springer Cham, 2December 2021. isbn: 978-3-030-81026-9. doi: https ://doi.org/10.1007/978-3-030-81026-9.[14]

2 What is Dimensionality Reduction?

Having established the appropriate context for data science in the preceding section, we can now delve into the topic at hand, dimensionality reduction (DR).

2.1 High Dimensional Data

High dimensional or Multidimensional data, also known as feature data, refers to a collection of n data points, or feature vectors, denoted as $\mathbf{x}_i$ in a high-dimensional vector space. Such datasets can be organized into a data matrix of dimensions $n \times d$. This format lends itself to the representation of data tables [14], where rows correspond to individual instances and columns are associated with specific features of those instances [14]. A sample $\mathbf{X}$ can be considered as a table containing n observations (with $n > 1$), each expressed in vectors $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)$, where $\mathbf{x}_i = (x_{i1}, \ldots, x_{id})$ represents the covariate vector of input features for the ith observation, and $\mathbf{y}_i$ signifies the response variable [5]. Thus, the data matrix can be visually represented as

$$\mathbf{X}_{nd} = \begin{bmatrix} x_{11} & \ldots & x_{1d} \\ x_{21} & \ldots & x_{2d} \\ \vdots & \ddots & \vdots \\ x_{n1} & \ldots & x_{nd} \end{bmatrix} = \begin{bmatrix} X_1, & X_2, & \ldots, & X_d \end{bmatrix}, \mathbf{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix},$$

Where the input matrix can be regarded as comprising either n row vectors $(\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n)$, each of dimension d representing the data points, or d column vectors $(X_1, X_2, \ldots, X_d)$, each of dimension n representing the features. Additionally, $\mathbf{Y}$ denotes the response column vector of dimension n [5].

2.2 Dimensionality Reduction

In general, Dimensionality Reduction (DR) aims to discover lower-dimensional representations of data while retaining their essential characteristics for a given problem either linearly or non-linearly [6]. DR is especially effective in managing big data that contains many correlated input variables.[5]. The conventional approach presupposes the solution to a given problem relies on a function f which involves d covariates in $\mathbf{X}$, yielding n-dimensional response vectors $\mathbf{Y}$, where d is too large for efficiently approximating f. Often, the columns within $\mathbf{X}$ show strong correlations, or certain columns might exhibit nonlinear relationships. As a result, the number of features can be reduced to p ($p \ll d$). This reduction can be seen as a transformation Ψ that converts the original dataset $\mathbf{X}$ into a new matrix $\mathbf{X}^*$ consisting of p columns,

$$\Psi(\mathbf{X}) \equiv \mathbf{X}^* \equiv [X_1^*, X_2^*, \ldots X_p^*]. \tag{1}$$

This reduced set of features could be employed to solve the problem of predicting the response based on the features, establishing a relationship between $\mathbf{Y}$ and $\mathbf{X}^*$ expressed as

$$\mathbf{Y} = f(\Psi(\mathbf{X}^*)) + \epsilon, \tag{2}$$

where ϵ is a random error term and f is the response function.

Due to the intricate nature of big data, it is frequently necessary to employ DR techniques prior to embarking on statistical inference or solving a problem. Some of these techniques will be discussed later in sections 3 and 4. It is noteworthy that the methods provided might be suitable for one task but unsuitable for another, owing the presence of tunable parameters that make specific assumptions. Consequently, the efficacy of the model could significantly depend on the fine-tuning of these parameters [5].

The references used for this section:

Max Garzon et al. Dimensionality Reduction in Data Science. Springer Cham, 28 July 2022. isbn: 978303105373. doi: https://doi.org/10.1007/978-3-031-05371-9.[5]

Sylvain Lespinats, Benoit Colange, and Denys Dutykh. Nonlinear Dimensionality Reduction Techniques. Springer Cham, 2December 2021. isbn: 978-3-030-81026-9. doi: https ://doi.org/10.1007/978-3-030-81026-9.[14]

3 DR Linear Methods

In the upcoming sections, we will delve into dimensionality reduction techniques that employ a geometric strategy, known as *Spectral Dimensionality Reduction* methods [6]. These methods aim to identify the linear and non-linear sub-manifolds within the data, typically reduced to a generalized eigenvalue problem [6]. A linear method relies on the Euclidean distances among points in a space, indicating its potential limitations with non-linear data [6]. Hence, we will apply a well-known dataset, the *Iris Dataset* [4], to illustrate the effectiveness of two geometric linear dimensionality reduction methods: *Principal Component Analysis (PCA)* and *Linear Discriminant Analysis (LDA)*.

3.1 Iris Dataset

The Iris dataset is a data set that contains 150 different iris flowers among three species (Setosa, Versicolour, and Virginica) with four numerical attributes: sepal length, sepal width, petal length, and petal width [4]. Hence, the iris dataset is of 4-dimension. Specifically, these attributes, or so-called covariates, allow us to predict the species of an iris flower. We will import the Iris dataset into Python to utilize its functions.

General statistical information on the Iris dataset is shown in Figure1.

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Figure 1: Basic statistics of iris data using Python

We can deduce how the covariates correlate pairwise by generating a correlation matrix (Figure 2).

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
SepalLengthCm	1.000000	-0.109369	0.871754	0.817954
SepalWidthCm	-0.109369	1.000000	-0.420516	-0.356544
PetalLengthCm	0.871754	-0.420516	1.000000	0.962757
PetalWidthCm	0.817954	-0.356544	0.962757	1.000000

Figure 2: Correlation Matrix using Python

There are strong correlations between petal length and petal width, petal length and sepal length, as well as petal width and sepal length (Also shown in Figure 3).

Figure 3 shows the correlation between each two covariates. The redder the grid is, the stronger the correlation is.

The aim of dimensionality reduction is to capture the information we just computed with fewer components. Either by deleting covariates or by computing new components based on the relation of original covariates. Different dimensionality reduction methods focus on different information

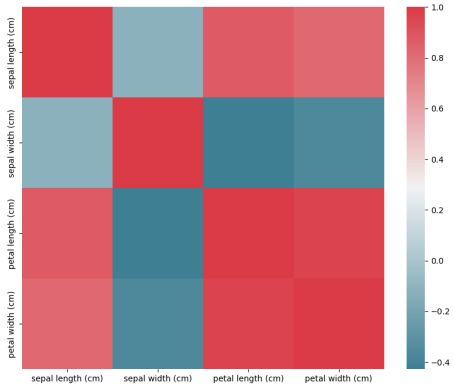


Figure 3: Colored Corr Matrix (left) and Pair Plot (right)

aspects of the dataset. For example, Principal Component Analysis concentrates on the variance of data, while Linear Discriminant Analysis divides the group clearly.

A pair plot can further present this.

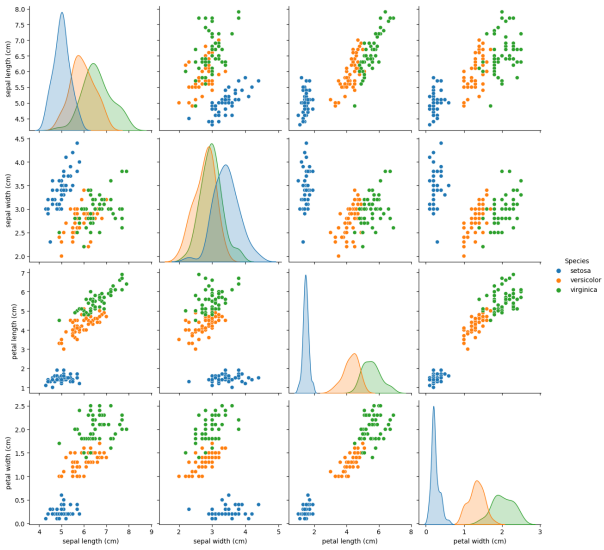


Figure 4: Pair Plot

The plots on the diagonal line of Figure 4 show the distribution of different groups on the specific covariate. When we are curious about the variance of the data, it is useful to concentrate on those or some combination of those covariates with a large variance distribution. The rest of the plots are the distribution of samples with each two covariates to be the x-axis and y-axis. We can see from the graphs that the stronger the correlation is, the clearer groups are separated using these two covariates. Hence, when we want to separate the groups clearly, we can focus on the covariates with strong correlations. These are different aspects we are concerned about when choosing dimensionality reduction methods.

In the following sections, The Principal Component Analysis, Linear Discriminant Analysis, and Kernel Principal Component Analysis will be used to reduce the dimensionality of the Iris dataset from 4-dimensional space to 2-dimensional space.

3.2 Principal Component Analysis

3.2.1 Introduction to PCA

Main Idea of PCA Principal Component Analysis (PCA) is to abstract the raw dataset into a new set of fewer components that have the largest variance.

Principal Component Principal components (PCs) are linear combinations of the covariates of the original dataset. They are chosen to have the largest variance; in this method, we take variance

as a presence of information. This is because the more spread the data is, the easier one can deduce the potential relationship in the dataset. Hence, the spread out datasets clearly facilitates a more comprehensive understanding of the data when compared to the clustered ones. This way, groups can be distinguished, or regressions can be fitted and so-called containing information. Furthermore, PCs must also achieve pairwise uncorrelated, i.e. $cov(PC_i, PC_j) = 0$ for any $i \neq j$.

3.2.2 Understand PCA with Python Function

We first use the preset package in Python for the iris data introduced before to gain a basic idea of the PCA method. By applying Python's PCA function, we can fit four PCs and compute their explained variance, variance ratio and cumulative variance ratio (shown in Figure 5).

```
Explained variance of each component [4.22824171 0.24267075 0.0782095  0.02383509]
Explained variance ratio (over sum) of each component: [0.92461872 0.05306648 0.01710261 0.00521218]
Cumulative variance ratio among each component: [0.92461872 0.97768521 0.99478782 1.        ]
```

Figure 5: Variance of PCs

The four PCs are ordered in descending variance due to the definition of taking the largest variance. We use a line chart to visualise the cumulative variance ratio in Figure 5.

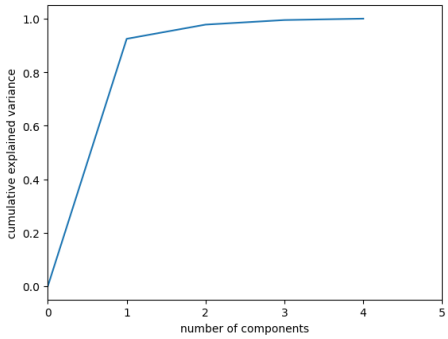


Figure 6: Cumulative Var Ratio using Python

We can see from Figure 6 that the cumulative variance ratio is 98% and does not significantly increase after the number of components reaches two. From this, we deduce that the dimensionality of the iris dataset can be reduced from four to two, which means that we only need two covariates (PCs) to obtain sufficient information to classify an iris flower.

Computing a 2D plot allows us to know how the first two PCs explain the data. We can see

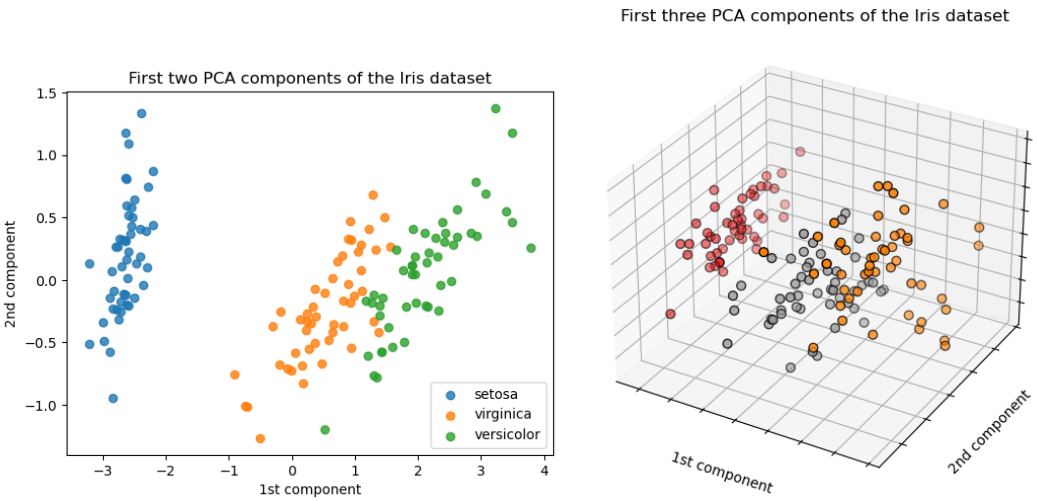


Figure 7: First two and three PCs of the Iris Dataset

in the left plot of Figure 7 that the data is already separated among the x-axis and has a little

difference among the y-axis. This is because PC1 have a 92% explained variance ratio, and PC2 only adds 5.3% to it. It gets worse when it comes to a three-dimensional plot, as there is almost no distinction among the z-axis. Hence, we can conclude that it is a waste to use three PCs to predict the species of an iris flower and enough with two. This is how we use a PCA method to reduce the dimensionality of a dataset.

The PCA method can not only be gained by using Python but can be computed manually as well. The next section will introduce how the principle components are calculated by applying linear algebra.

3.2.3 Compute Principal Components by Math

Notation of Principal Components Recall that in the Section 3.2, PCs are linear combinations of the covariates of the dataset. Let x be a vector taking values in $\mathbb{R}^p$,

$$x\alpha = \alpha_1 x_1 + \alpha_2 x_2 + \cdots + \alpha_p x_p = \sum_{i=1}^{p} \alpha_i x_i \tag{3}$$

where x is an observation of the dataset X and $\alpha \in \mathbb{R}^p$. Similarly, PCs of a dataset with n observations are $X\alpha$ where X is a $n \times p$ matrix.

The Variance of PCs Hence, we can deduce the variance of PCs by the following theorem.

Proposition 1. *Let $cov(X)$ denote the covariance matrix of the dataset X, then the variance of PCs $var(X\alpha) = \alpha^T cov(X)\alpha$*

Proof of the proposition.

$$\begin{aligned}
var(X\alpha) &= \mathbb{E}((X\alpha)^T(X\alpha)) - \mathbb{E}(X\alpha)^2 && \text{(Definition of variance)} \\
&= \mathbb{E}(\alpha^T X^T X\alpha) - \mathbb{E}(\alpha^T X^T)\mathbb{E}(X\alpha) \\
&= \alpha^T \mathbb{E}(X^T X)\alpha - \alpha^T \mathbb{E}(X^T)\mathbb{E}(X)\alpha && (\mathbb{E}(\alpha) = \alpha) \\
&= \alpha^T (\mathbb{E}(X^T X) - \mathbb{E}(X^T)\mathbb{E}(X))\alpha && \text{(Extract common factors)} \\
&= \alpha^T cov(X)\alpha
\end{aligned}$$

□

As we are looking for principal components in a variance descending order, we let $X\alpha_1$ denote the first PC with the largest variance.

Find the Largest Variance Finding a largest $var(X\alpha)$ is equivalent to finding the largest $\alpha^T cov(X)\alpha$. However, as you may notice, maximising this product does not have a finite solution. Therefore, we need a limitation for the vector α.

Proposition 2. *α is a unit vector in $\mathbb{R}^p$, that is $\alpha^T\alpha = 1$.*

There are many other conditions we can make to restrict the solution. But here, to simplify the computation, we choose to let the norm equal to 1. We then compute the maximum by introducing a Lagrange multiplier λ.

Proposition 3. *Finding the maximum of $\alpha^T cov(X)\alpha$ is equivalent to finding the maximum of*

$$\alpha^T cov(X)\alpha - \lambda(\alpha^T\alpha \tag{4}$$

where $\lambda \in \mathbb{R}$.

Proof. From Proposition 2, we have $\alpha^T\alpha - 1 = 0$. Thus,

$$\alpha^T cov(X)\alpha = \alpha^T cov(X)\alpha - 0 = \alpha^T cov(X)\alpha - \lambda(\alpha^T\alpha - 1)$$

□

Taking the derivative of equation 4 and letting that equal to 0, we gain the maximum of the variance and the first PC $X\alpha_1$.

$$cov(X)\alpha_1 - \lambda\alpha_1 = 0 \tag{5}$$

We can, therefore, deduce that λ, α_1 is a pair of eigenvalue and eigenvector of matrix $cov(X)$, with λ the spectral radius (the largest absolute eigenvalue) of $cov(X)$. Similarly, we can compute all the other PCs to be the eigenvectors of $cov(X)$ in descending order among the absolute values of their corresponding eigenvalues.

Covariance between PCs Recall that another assumption besides the variance for the principal components is that they are pairwise uncorrelated. Hence, in the last step, we are going to show that if PCs take values in the eigenvector of the covariance matrix, the covariance between any two of them is zero.

$$\begin{aligned} cov(X\alpha_i, X\alpha_j) =& \frac{1}{n}\sum_{l=1}^{n} \alpha_i^T \bar{X}_l \alpha_j^T \bar{X}_l \\ =& \frac{1}{n}\sum_{l=1}^{n} \alpha_i^T \bar{X}_l \bar{X}_l^T \alpha_j \\ =& \alpha_i^T (\frac{1}{n}\sum_{l=1}^{n} \bar{X}_l \bar{X}_l^T)\alpha_j \qquad \text{(Extract common factors)} \\ =& \alpha_i^T R \alpha_j \end{aligned}$$

Applying the eigenvalue decomposition to the equation. Let $e_m \in \mathbb{R}^p$ $m = 1, 2, \ldots, p$ be a base of $\mathbb{R}^p$, where $[e_m]_i = 0$ for $i \neq m$ and $[e_m]_m = 1$. Hence,

$$\alpha_i^T R \alpha_j = e_i^T diag(d_i^2, \ldots, d_p^2) e_j = 0$$

Thus, the covariance between any two $X\alpha_i$ and $X\alpha_j$ is zero. Therefore, linear combinations in the form $X\alpha$ where α is the eigenvector of $cov(X)$ are principal components of the PCA method.

The references used for this section:

Jolliffe, I. T. (2002) Principal component analysis. 2nd edn. New York: Springer (Springer series in statistics). [9]

3.3 Linear Discriminant Analysis

3.3.1 Introduction to Linear Discriminant Analysis

In advancing data analysis methodologies, particular attention is warranted by Linear Discriminant Analysis (LDA). LDA commands interest as the discourse progresses from general analytical strategies due to its distinctive capacity to determine and utilize the most discriminative features for class differentiation. This method simplifies the complexity of data by reducing its dimensions and methodically capitalizing on predefined class labels. Unlike other dimensionality reduction techniques that reduce data complexity without regard to class outcomes, LDA optimizes the feature selection process to bolster the model's classification accuracy significantly. Thus, LDA enhances the suite of statistical tools by providing a refined approach to feature reduction driven by the imperatives of classification efficacy [3, 11].

This methodology aims to improve the separation of different classes within a dataset. Contrasting with Principal Component Analysis (PCA), which prioritizes variance regardless of class labels, LDA seeks axes that maximize the separation between multiple known classes [13]. This property of LDA is beneficial for classification purposes and makes it an invaluable tool for reducing dimensions in a way sensitive to the underlying class structure [11].

3.3.2 Theoretical Foundations of LDA

Gaussian Distribution Assumption A foundational aspect of LDA is its reliance on the assumption that the data within each class follow a Gaussian distribution. Each class is characterized by a unique mean while sharing a common covariance matrix with other classes. This assumption not only aligns with the Gaussian distribution's prevalence in modelling a wide array of real-world phenomena but also significantly simplifies the computation of discriminant functions. The Gaussian distribution, with its characteristic bell-shaped curve, symmetrically distributes data around the mean, thereby facilitating a natural and efficient model for feature spaces in LDA [10, 8].

Homoscedasticity Assumption A crucial assumption underpinning LDA is homoscedasticity, which posits that the covariance matrices across all classes are identical. This assumption of uniform covariance simplifies the LDA model significantly, enhancing its computational efficiency and setting

it apart from other discriminant analysis techniques that allow for class-specific covariances. While this simplification facilitates model tractability and calculation, it is essential to recognize its potential limitations. The homoscedasticity assumption may not always hold true for every dataset, and its validity directly impacts the effectiveness of LDA. Recognizing when this assumption is applicable is vital for the successful application of LDA [8].

3.3.3 LDA Concept and Methodology

At its core, LDA operates on a fundamental principle: finding the linear combinations of variables that best separate different classes. This statistical method integrates the Gaussian distribution assumption to model data within each class, relying on mean vectors and a shared covariance matrix across all classes. The essence of LDA lies in its ability to effectively utilize these statistical measures to distinguish between classes [5, 14].

Central to the LDA's methodology is the construction of a decision rule that maximizes the separation between classes. This is achieved by projecting the data onto a new axis to maximize the distance between the means of different classes while minimizing the variance within each class. The result is a new feature space where classes are more linearly separable, facilitating easier classification and reducing the dimensionality of the data with minimal loss of information, which is crucial for classification [14, 12, 3, 10].

LDA's discriminative power is further refined by computing two fundamental matrices: the within-class scatter matrix (S_W) and the between-class scatter matrix (S_B). The former measures the variance within each class around its mean, highlighting the compactness of the class distribution. The latter assesses the spread of the class means themselves, providing insight into the classes' separability in the original feature space. By optimizing the ratio of the determinant of these two matrices, LDA finds the most effective linear discriminants that serve as the new axes for the reduced-dimensional space [14, 12, 3, 10].

3.3.4 Mathematical Formulation of LDA

The Mathematical Formulation of LDA delves into the theoretical underpinnings that enable LDA to effectively differentiate between classes in a dataset. This formulation is based on solving an optimization problem to find the projection that maximizes the between-class and within-class variance ratio.

Essence of LDA The essence of LDA lies in its mathematical formulation, which seeks a projection that optimally separates classes within the feature space. This is approached as an optimization problem with the objective of maximizing the ratio of the between-class scatter matrix (S_B) to the within-class scatter matrix (S_W). The essence of this optimization can be mathematically expressed as maximizing the criterion $\frac{|S_B|}{|S_W|}$, where $|S_B|$ and $|S_W|$ denote the determinants of the between-class and within-class scatter matrices, respectively [14, 12, 3].

Calculating Class Means and Scatter Matrices To achieve this maximization, for a dataset comprising C classes and N instances, LDA calculates the mean vector of each class and the two scatter matrices as follows:

1. Calculate the mean vector of each class (μ_c): This represents the centroid of the instances belonging to that class. It is computed as the average of all feature vectors within the class:

$$\mu_c = \frac{1}{N_c} \sum_{i \in c} x_i$$

   Here, N_c is the number of instances in class c, and x_i represents the feature vector of the ith instance in that class [3, 11].

2. Calculate the within-Class Scatter Matrix (S_W): This represents the measure of how dispersed the data points are within each class, computed as:

$$S_W = \sum_{i \in c} (x_i - \mu_c)(x_i - \mu_c)^T$$

   In this formula, x_i is the feature vector of instance i in class c, and μ_c is the class mean vector. The within-class scatter matrix S_W is obtained by summing these individual class covariance matrices to measure the overall variability within each class [3, 11].

3. Calculate the between-class scatter matrix (S_B): This is computed to assess the separation between different classes. This matrix measures how far apart the different class means are from each other, and it is defined as:

$$S_B = \sum_{c=1}^{C} N_c(\mu_c - \mu)(\mu_c - \mu)^T$$

Here, μ is the overall mean vector of the dataset, representing the centroid of all instances across all classes, calculated as the mean of all class means:

$$\mu = \frac{1}{C}\sum_{c=1}^{C} \mu_c$$

[3, 11]

By optimizing the ratio $\frac{|S_B|}{|S_W|}$, LDA seeks to find the projection direction(s) that enhance class separability. This involves solving for the eigenvalues and eigenvectors of the matrix $S_W^{-1}S_B$, where the eigenvectors represent the directions for the new feature space that maximizes the distance between class means (captured by S_B) while minimizing the variance within each class (captured by S_W). This optimization underscores LDA's capability to reduce the data's dimensionality and prioritize the most discriminative features for class differentiation. The process ensures that the transformation of the data into a lower-dimensional space preserves the essential characteristics required for accurate classification, thus underscoring the method's robustness in enhancing classification efficacy [14, 12, 3, 11].

3.3.5 Application of LDA to the Iris Dataset

This section will demonstrate the practical application of LDA using the renowned Iris dataset. The implementation will be conducted using Python.

Before applying LDA to the Iris dataset, it is crucial to preprocess the data to ensure optimal performance. This preparation involves two key steps:

Step 1: Data Cleaning Although the Iris dataset typically requires minimal data cleaning, the process of preparing any dataset for analysis must include a thorough examination for missing values. Handling missing data is crucial to maintain the integrity of the analysis. The following strategies can be employed depending on the scenario:

1. **Imputation:** Replacing missing values with statistical measures such as the mean or median of the respective feature. This method is useful when the missing data is assumed to be randomly distributed and the total percentage of missing values is not significantly high.
2. **Deletion:** Removing records with missing values. This approach is considered when the missing data is minimal, and its absence does not skew the dataset's overall distribution.
3. **Estimation:** Utilizing machine learning models to estimate and fill in missing values based on the information available in the dataset. This method is more sophisticated and can be beneficial when patterns within the data can predict the missing values accurately.

It is important to carefully choose the most appropriate method for handling missing data, considering the nature of the dataset and the objectives of the analysis.

Step 2: Normalization The features are scaled to achieve zero mean and unit variance. This normalization process is pivotal, preventing the dominance of any single feature during the analysis and ensuring uniform contribution across all features to the LDA computation. This step is especially critical in LDA, where equal scaling of features contributes to a more accurate computation of covariance matrices, ultimately affecting the classification and dimensionality reduction performance.

With the dataset now preprocessed, the stage is set for applying LDA for dimensionality reduction. The aim is to project the Iris dataset into a lower-dimensional space where the separability between different species, based on features like sepal length and petal width, is maximized, thereby facilitating a clearer visualization and understanding of the data's inherent structure.

Step 3: Compute Class Means and Scatter Matrices Calculate the mean vectors for each class and the within-class scatter matrix (S_W) and between-class scatter matrix (S_B) based on the preprocessed data.

The computed class means are as follows:

- Class 0 mean: $[-1.01, 0.85, -1.30, -1.25]$
- Class 1 mean: $[0.11, -0.66, 0.29, 0.17]$
- Class 2 mean: $[0.90, -0.19, 1.02, 1.09]$

These means indicate the central tendency of each class in the multidimensional feature space, which is essential for understanding the class's distribution. The within-class scatter matrix ($\mathbf{S}_W$) and the between-class scatter matrix ($\mathbf{S}_B$) are calculated as:

$$\mathbf{S}_W = \begin{bmatrix} 57.19 & 38.02 & 16.96 & 9.00 \\ 38.02 & 89.88 & 10.63 & 14.57 \\ 16.96 & 10.63 & 8.79 & 4.69 \\ 9.00 & 14.57 & 4.69 & 10.67 \end{bmatrix}$$

$$\mathbf{S}_B = \begin{bmatrix} 92.81 & -55.65 & 113.80 & 113.69 \\ -55.65 & 60.12 & -74.89 & -69.49 \\ 113.80 & -74.89 & 141.21 & 139.74 \\ 113.69 & -69.49 & 139.74 & 139.33 \end{bmatrix}$$

The within-class scatter matrix (S_W) reflects the spread of the data within each class. If this scatter is small (i.e., the matrix values are small), it indicates that data points in each class are closely bunched together. A lower within-class scatter is generally desirable because it implies that members of each class are similar, making the class more distinct.

Conversely, the between-class scatter matrix (S_B) reflects the separation between the different class means. Higher values in this matrix indicate that the class means are further apart, which signifies that the classes are well-separated in the feature space.

From the matrices provided: The between-class scatter matrix (S_B) has larger absolute values than the within-class scatter matrix (S_W). This suggests a greater degree of separation between the classes since S_B reflects how far the means of different classes are quite distinct from the overall mean. This is desirable for discriminant analysis because it suggests that classes can be more easily separated. On the other hand, the values in S_W are comparatively smaller, implying that data points within each class are closer to their class mean. This is also favourable for discriminant analysis since it suggests that classes are more tightly clustered, with less overlap in the feature space.

When LDA utilizes these matrices to find the optimal linear discriminants, a smaller S_W in comparison to S_B generally leads to a projection where the classes are more distinctly separable. Thus, the provided values suggest good discriminatory power when applying LDA to this dataset.

Step 4: Compute Eigenvectors and Eigenvalues To identify the optimal projection for class separability, we solve the generalized eigenvalue problem defined by $\mathbf{S_W^{-1}S_B w} = \lambda \mathbf{w}$. This step is critical for determining the directions along which the separation between classes is maximized. The solution provides us with eigenvalues and corresponding eigenvectors:

Eigenvalues:

$$\begin{bmatrix} 3.2 \times 10^{1} & 0 \\ 2.9 \times 10^{-1} & 0 \\ 1.1 \times 10^{-15} & 2.1 \times 10^{-15} i \\ 1.1 \times 10^{-15} & -2.1 \times 10^{-15} i \end{bmatrix}$$

Eigenvectors:

$$\begin{bmatrix} 0.15 + 0.j & 0.0069 + 0.j & 0.21 - 0.51j & 0.21 + 0.51j \\ 0.15 + 0.j & 0.33 + 0.j & -0.13 + 0.021j & -0.13 - 0.021j \\ -0.86 + 0.j & -0.57 + 0.j & -0.60 + 0.j & -0.60 - 0.j \\ -0.47 + 0.j & 0.75 + 0.j & 0.37 + 0.43j & 0.37 - 0.43j \end{bmatrix}$$

The interpretation of eigenvalues and eigenvectors can be complex, particularly when they have non-zero imaginary parts. However, within the framework of LDA, the real part of these eigenvalues are of interest. The real parts signify the extent of variance each eigenvector encapsulates, which is

crucial in discerning the most substantial directions for data projection in feature space. In practice, the focus is on those eigenvectors that correlate to the most significant real eigenvalues, as they delineate the axes along which the data can be projected to maximize class separation.

The real component of the first eigenvalue, quantified as 3.2×10^{1}, denotes a significant share of the variance between classes being accounted for by its associated eigenvector. Conversely, the second eigenvalue, noticeably lesser at 2.9×10^{-1}, reflects a reduced variance captured by its corresponding eigenvector, albeit still considerable compared to the subsequent two eigenvalues. These latter eigenvalues, approaching zero and accompanied by complex parts, indicate minimal variance capture, a scenario often resulting from computational rounding and the intrinsic limitations of floating-point precision, underlining their negligible contribution to the LDA's discriminative objective.

Step 5: Select Discriminant Directions Selecting discriminant directions in LDA involves choosing eigenvectors linked to the largest eigenvalues, as these capture the most variance between classes, guiding the projection into a space where classes are more separable.

LDA components:

$$\begin{bmatrix} 0.15 & 0.01 \\ 0.15 & 0.33 \\ -0.86 & -0.57 \\ -0.47 & 0.75 \end{bmatrix}$$

The LDA components matrix, formed from these significant eigenvectors, outlines the directions for this projection. Each column in this matrix represents a discriminant direction, crucial for enhancing class separability when projecting high-dimensional data into a reduced-dimensional space. This step is fundamental for maximizing class distinguishability in the transformed feature space.

Step 6: Project Data onto the Discriminant Directions In the final step, we project the Iris dataset into the most informative directions, transforming it into a simpler space that still captures essential class differences. This transformation uses selected eigenvectors, creating a new dataset with reduced dimensions. The transformation of the first five samples, illustrated by the following matrix, exemplifies how each data point is now positioned in this new, simplified space.

Projected data (first 5 samples):

$$\begin{bmatrix} 1.78 & 0.10 \\ 1.58 & -0.27 \\ 1.66 & -0.09 \\ 1.51 & -0.23 \\ 1.80 & 0.18 \end{bmatrix}$$

This step highlights LDA's power in making classes more distinguishable, facilitating better visualization and more effective classification in a space optimized for highlighting differences between groups.

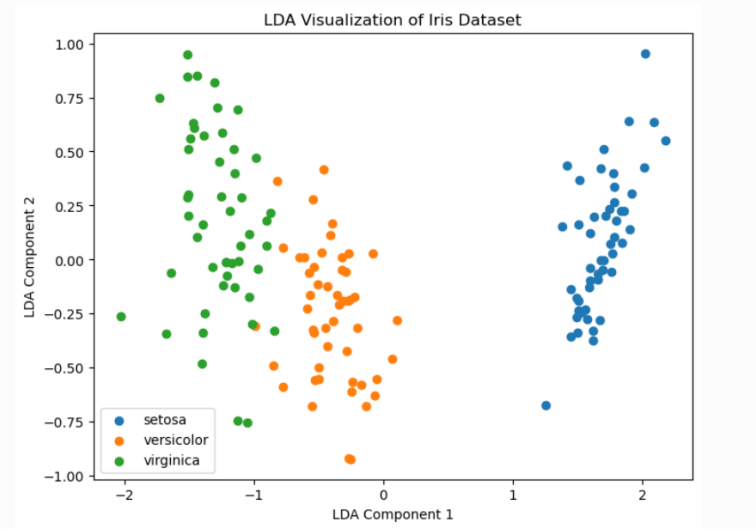


Figure 8: LDA Visualization of Iris Dataset

Visualize the reduced-dimensional space The plot shows three distinct clusters, each representing one of the three species of the Iris flower: setosa (in blue), versicolor (in orange), and virginica (in green). The clear separation of the setosa species along the LDA Component 1 axis indicates that this species has distinct petal and sepal characteristics compared to the other two species. The versicolor and virginica species are closer together but still form distinct clusters, suggesting that while they share more similar characteristics than either does with setosa, they are still separable based on the linear combinations of features captured by LDA.

LDA Component 1: This axis is the most significant in terms of explaining the variance between the species. The separation along this axis is quite pronounced for setosa, indicating that the corresponding eigenvector captures features that strongly differentiate setosa from the other two species.

LDA Component 2: This axis contributes to the separation to a lesser extent compared to Component 1. It appears to assist in distinguishing between versicolor and virginica, which are not as separated along Component 1.

The effectiveness of LDA as a dimensionality reduction technique prior to classification is evident. The resulting two-dimensional space should facilitate the use of simple linear classifiers to achieve high classification accuracy, especially for setosa, which is linearly separable from the other two species. For versicolor and virginica, while there is some overlap, a linear classifier might still perform well, given the overall clustering.

4 DR Non-Linear Methods

In the following sections, we will introduce two non-linear methods: *Isometric Mapping (Isomap)*, and *Kernel Principal Component Analysis (KPCA)*.

4.1 Isometric Mapping (Anas's section: not complete)

4.1.1 Multi-Dimensional Scaling (MDS)

Introduction The aim of MDS is to find a p-dimensional sub-space such that the Euclidean distances between the data points in the sub-space $\mathbb{R}^p$ is preserved as much as possible comparing to the distance between the data points in the input space (the original space before reducing the dimensions). To understand the aim of MDS better, we will consider the following example: For any three points in $\mathbb{R}^3$, we can find a plain that passes through the three point, so it is obviously possible to find three point in $\mathbb{R}^2$ such that the distance between the points in $\mathbb{R}^3$ is exactly equal to the distances between the points in $\mathbb{R}^2$. However, If we add a fourth point that does not lie on the same plain, it will be impossible to do what we did in the case of the three point. However, we can still represent our data in $\mathbb{R}^2$ with minimal losses in the sense that we minimize the difference between all the possible distances between the points in $\mathbb{R}^3$ and their corresponding distances in $\mathbb{R}^2$.

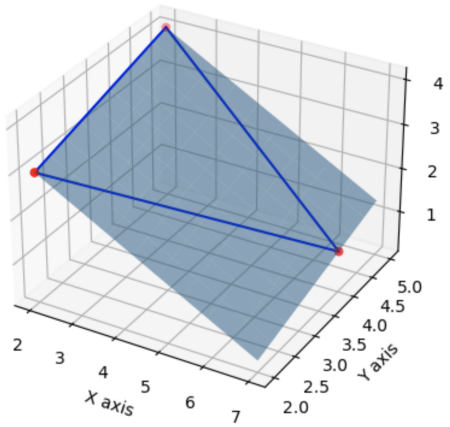


Figure 9: This is done using Python

The aim of MDS instead of Euclidean distance, we will use inner product to express similarity

between the data points. Our aim then is to minimize the following:

$$\sum_i \sum_j \left(x_i^{\mathrm{T}} x_j - y_i^{\mathrm{T}} y_j\right)^2$$

In the above expression, x_i is the ith feather vector in the input space, and y_i is the corresponding feather vector in the embedded space (the reduced space). So the expression above sums up the square of the differences between the inner product of all possible selections of two points and the inner product of the two corresponding points. We can write the above formula in matrix form as follows:

$$\|XX^{\mathrm{T}} - YY^{\mathrm{T}}\|^2_{\mathrm{Frobenius}}$$

$(XX^T)_{ij}$ is n×n matrix, where n is the number of data points. The i,j entries of $(XX^T)_{ij}$ represent the inner product between x_i and x_j. The entries for $(YY^T)_{ij}$ must be obvious. When subtracting the two matrices from each other, the resulting matrix represent the differences in inner products that we want to minimize when squared and summed. It should be clear that the Frobenius norm is the square root of the sum of squares of all the entries in the matrix.

4.1.2 Isometric Mapping (Isomap)

Isomap is non-linear dimensionality reduction method which is a generalization of MDS. Ineasted of using the Euclidean distance, it uses what so called geodesic distance, which is the shortest distance between two points through a manifold (could be a curvy plain in the case of $\mathbb{R}^3$).

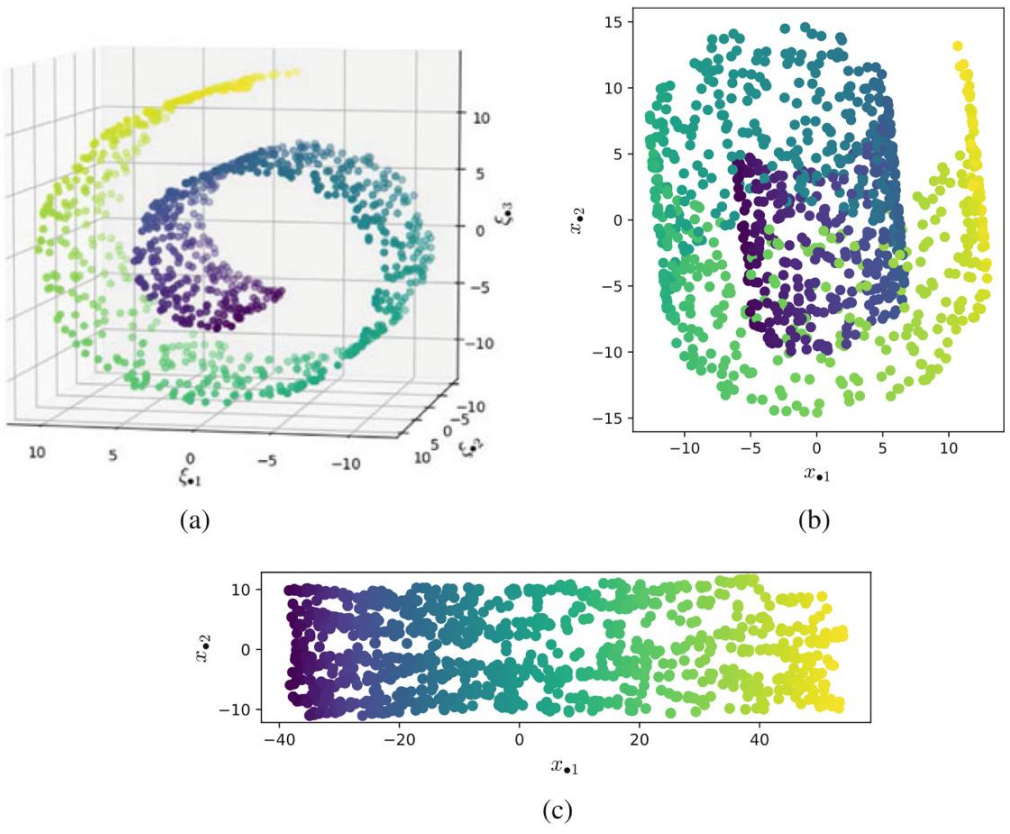


Figure 10: (c) shows the Isomap embedding of the roll (b) PCA projections

Example In Figure 10a, the geodesic distance between any two points is the shortest path through the "roll" formed by the data. Notice that if we embed the data in a two dimensional space, Figure 2.c, the geodesic distance will be preserved, and it can be calculated by finding the Euclidean distance between the points [6].

4.2 KPCA

4.2.1 Kernel Method

Recall that we gain the principal components in the linear PCA method by linear combinations of the original covariates. However, the covariates of real-life data do not always have a linear relation. Consider the following Example.

Example 1. *Data with an oval boundary*

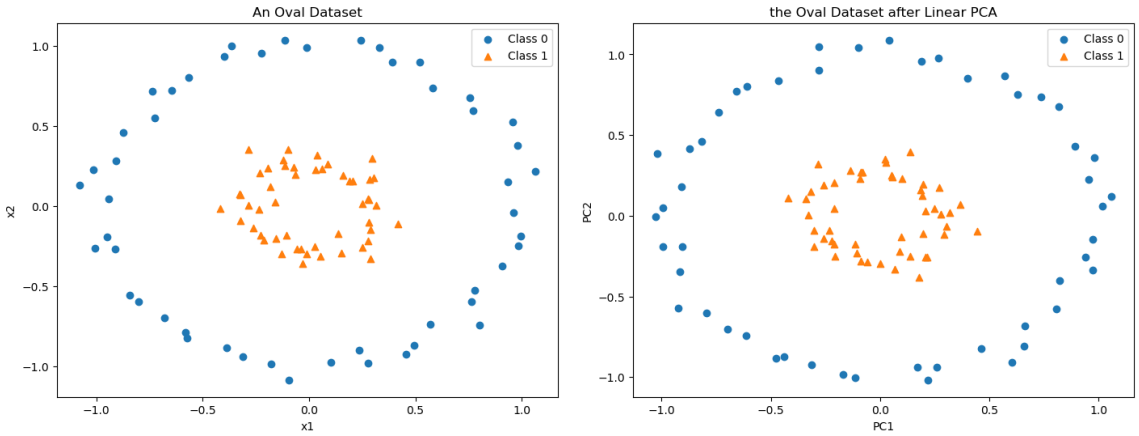


Figure 11: An oval dataset and its linear PCA

In the left plot of Figure 11, we can see that there is no linear relationship between the covariates x_1 and x_2, meaning that no single line can be figured out to separate the two classes of points. In such cases, we may attempt to use PCA to extract linear combinations of the covariates x_1 and x_2 that capture the maximum variances. After performing PCA and extracting two principal components, the result, as shown in the right plot of Figure 11, indicates no significant improvement at this stage. Here, linear dimensionality reduction encounters setbacks. This problem is common in real-life scenarios, where the original covariates do not exhibit a linear relationship, and relying only on linear combinations yields poor results. To solve this, we introduce the concept of a feature space.

Feature Space A feature space is a higher-dimensional linear space, where we can utilize a feature space mapping, ϕ, to project the original data to the feature space. $\phi : \mathbb{R}^p \to \mathbb{R}^d$ where p is the original dimension of dataset X and d is a higher dimension. The feature space is chosen in order to find a linear relationship in its new components while the raw dataset does not. For instance, in this case, if we let this mapping to be $\phi(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$, we can turn the original covariates (x_1, x_2) to be (ϕ_1, ϕ_2, ϕ_3). The feature space of the Example 1 is shown in Figure 12.
From Figure 12, we can deduce a plane separating the two classes' data points (perpendicular to the plane where the report lies). On its orthogonal planes, we can capture the data points with a large variance, which carries the largest variance and most information, overcoming the difficulty of linear methods on original space in extracting information effectively. Ironically, in a report dedicated to dimensionality reduction, we discuss increasing dimensions from two to three. This apparent contradiction is resolved by the magical Kernel Trick, which will be introduced later, which allows us to map the data to a higher-dimensional feature space without increasing the dimensionality of the covariates.

When analysing data, we often focus on the distance, angle and correlation between samples. For instance, when classifying a sample according to a training dataset, we can make the decision by comparing the distance between the new sample to the centres of existence groups. Furthermore, the distance and angle can be shown by the dot product of samples. In applications, we care more about the dot product between samples instead of their value.

Kernel Therefore, we utilize the dot product to construct the definition of a kernel.

Definition 2. *Kernel Function*

$$k(\mathbf{x}_1, \mathbf{x}_2) = \langle \phi(\mathbf{x}_1), \phi(\mathbf{x}_2) \rangle = \phi(\mathbf{x}_1)^T \phi(\mathbf{x}_2)$$

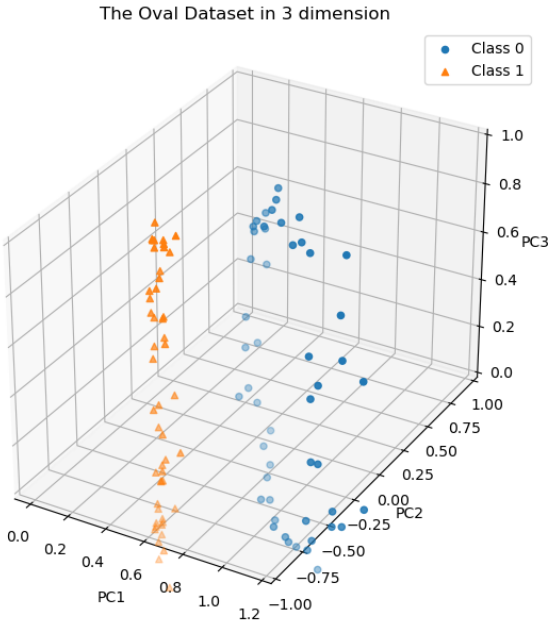


Figure 12: The oval dataset in 3D feature space

Kernel Trick The Kernel Trick is the advantage of using a kernel method. According to the Definition 2, we can use the kernel function to represent the dot product in the feature space. Since the result of the dot product is always a constant, the image of kernel function $k(\mathbf{x}_1, \mathbf{x}_2)$ is one-dimensional. This enables us to access the characteristics of high-dimensional feature space without increasing the dimensionality of the data. Furthermore, the kernel function can be obtained only by the dot product of the original data without the feature function ϕ. This allows us to completely disregard the specific mapping ϕ and focus directly on the kernel function. We will illustrate this with an example.

Example 2. *Kernel Function*

Considering the previous case, we mapped the oval dataset to a 3D feature space. The mapping function is

$$\phi(\mathbf{x}) = (\phi_1, \phi_2, \phi_3) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

Hence, the kernel method is

$$\begin{aligned} k(\mathbf{x}_1, \mathbf{x}_2) &= \langle \phi(\mathbf{x}_1), \phi(\mathbf{x}_2) \rangle \\ &= \langle (x_{11}^2, x_{12}^2, \sqrt{2}x_{11}x_{12}), (x_{21}^2, x_{22}^2, \sqrt{2}x_{21}x_{22}) \rangle \\ &= x_{11}^2 x_{21}^2 + 2x_{11}x_{12}x_{21}x_{22} + x_{12}^2 x_{22}^2 \\ &= (x_{11}x_{21} + x_{12}^2 x_{22})^2 \\ &= (\langle \mathbf{x_1}, \mathbf{x_2} \rangle)^2 \end{aligned}$$

In practical applications, there are numerous choices for feature functions and corresponding kernel functions. However, as long as there is a linear relation between components in the feature space, we can use the kernel function in the dimensionality reduction method.

4.2.2 KPCA method in Dimensionality Reduction

We can apply the kernel method to the principal component analysis to help us deal with nonlinear covariates.

Through the kernel method, we can transform data in the original space, which may not exhibit linear relationships, into data in the feature space with linear relationships. Thus, we can obtain the principal components of the original data by applying the PCA method to the feature space (i.e. the vector v_1 in the right plot of Figure 13). However, since we do not know the specific mapping

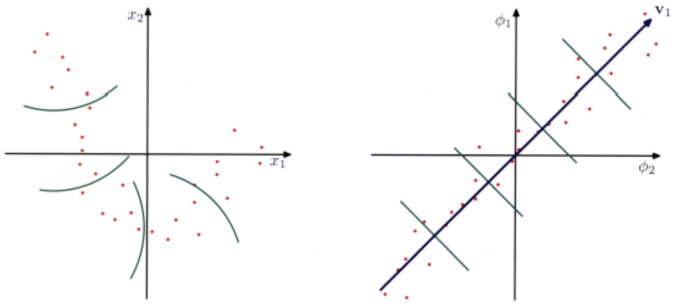


Figure 13: The Original and Feature Space of a Dataset[3]

ϕ, only the kernel function, establishing the relationship between the principal components in the feature space (i.e. linear combination of $\phi(\mathbf{x})$) and the kernel function becomes the primary focus of research on kernel functions. To facilitate computation, we use K to denote the kernel matrix.

Definition 3. *Kernel Matrix*
A kernel matrix K is defined as

$$\mathbf{K} = \{k(\mathbf{x}_i, \mathbf{x}_j)\}_{ij} = \{\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)\}_{ij} = \phi(\mathbf{X})\phi(\mathbf{X})^T$$

where

$$\phi(\mathbf{X}) = \begin{pmatrix} \phi(\mathbf{x}_1) \\ \phi(\mathbf{x}_2) \\ \vdots \\ \phi(\mathbf{x}_n) \end{pmatrix}$$

Recall the process of the PCA method introduced in Section 3.2. In order to gain the principal components in the feature space, we first need to compute its covariance matrix, denoted as $\mathbf{C}$. Here we assume that the data in the feature space $\phi(\mathbf{x})$ is already centralized. Hence, $\mathbf{C}$ can be represented as,

$$\mathbf{C} = \phi(\mathbf{X})^T \phi(\mathbf{X}) \tag{6}$$

Recall that in the linear PCA method, we proved that the principal components are in the form of $\mathbf{X}\mathbf{a}_i$ while $\mathbf{a}_i$, a unit vector, is the eigenvector corresponding to the i-th large eigenvalue of $cov(\mathbf{X})$. Similarly, we need to compute the eigenvector and eigenvalue of $\mathbf{C}$ for the kernel PCA. Without knowing the specific feature mapping ϕ, we need to build a connection between covariance matrix $\mathbf{C} = \phi(\mathbf{X})^T\phi(\mathbf{X})$ and kernel matrix $\mathbf{K} = \phi(\mathbf{X})\phi(\mathbf{X})^T$. Let $\mathbf{v}_i$ and λ_i denote the i-th eigenvector and eigenvalue of $\mathbf{K}$, and $\mathbf{v}_i$ is a unit vector. Hence, by the definition,

$$\mathbf{K}\mathbf{v}_i = \phi(\mathbf{X})\phi(\mathbf{X})^T\mathbf{v}_i = \lambda_i\mathbf{v}_i$$

We then multiply $\phi(\mathbf{X})^T$ to the left for both sides of the equation.

$$\begin{aligned} \phi(\mathbf{X})^T\phi(\mathbf{X})\phi(\mathbf{X})^T\mathbf{v}_i &= \phi(\mathbf{X})^T\phi(\mathbf{X})\phi(\mathbf{X})^T\mathbf{v}_i \\ &= \phi(\mathbf{X})^T\lambda_i\mathbf{v}_i && \text{(Definition of eigenvalue)} \\ &= \lambda_i\phi(\mathbf{X})^T\mathbf{v}_i && (\lambda_i \text{ is a constant, so we can put it at front}) \end{aligned}$$

Hence, $\phi(\mathbf{X})^T\mathbf{v}_i$ is an eigenvector of $\mathbf{C}$ with corresponding eigenvalue λ_i. Since $\phi(\mathbf{X})^T\mathbf{v}_i$ is not sure to be a unit vector, we need to normalize it. Denote the unit eigenvector as $\mathbf{u}_i$, we have

$$\begin{aligned} \mathbf{u}_i &= \frac{1}{\|\phi(\mathbf{X})^T\mathbf{v}_i\|}\phi(\mathbf{X})^T\mathbf{v}_i \\ &= \frac{1}{\sqrt{\mathbf{v}_i^T\phi(\mathbf{X})\phi(\mathbf{X})^T\mathbf{v}_i}}\phi(\mathbf{X})^T\mathbf{v}_i && \text{(Definition of vector norm)} \\ &= \frac{1}{\sqrt{\mathbf{v}_i^T\lambda_i\mathbf{v}_i}}\phi(\mathbf{X})^T\mathbf{v}_i && \text{(Definition of eigenvalue)} \\ &= \frac{1}{\sqrt{\lambda_i}}\phi(\mathbf{X})^T\mathbf{v}_i && (\lambda_i \text{ is a constant, so we can put it at front}) \end{aligned}$$

We still do not know the specific mapping $\phi(\mathbf{X})$ in the eigenvector equation. However, recalling that the principal components are linear combinations of the covariates, what we need is $\phi(\mathbf{X})\mathbf{u_i}$ instead of $\mathbf{u_i}$ for the PCs on feature space.

$$\begin{aligned}
\phi(\mathbf{X})\mathbf{u}_i &= \phi(\mathbf{X})\frac{1}{\sqrt{\lambda_i}}\phi(\mathbf{X})^T\mathbf{v}_i \\
&= \frac{1}{\sqrt{\lambda_i}}\phi(\mathbf{X})\phi(\mathbf{X})^T\mathbf{v}_i && (\lambda_i \text{ is a constant, so we can put it at front}) \\
&= \frac{1}{\sqrt{\lambda_i}}\mathbf{K}\mathbf{v}_i && (\text{Definition of kernel matrix } \mathbf{K}) \qquad (7)
\end{aligned}$$

Every part of Equation 7 is known as long as we know the kernel function k. There is one last step we need to consider. When computing the covariance matrix, we assumed the data in feature space to be centralized, which is not the common case. Thus, we need to consider what the covariance matrix really is and the kernel matrix $\tilde{\mathbf{K}}$ corresponding to the centralized data.

We use $\tilde{\phi}(\mathbf{X})$ to denote the centralized featured data. Hence, by the definition of centralization,

$$\tilde{\phi}(\mathbf{X}) = \phi(\mathbf{X}) - \frac{1}{N}\sum_{l=1}^{N}\phi(\mathbf{x}_l)$$

where N is the number of samples (i.e., the number of rows of matrix $\phi(\mathbf{X})$).

Centralized Kernel Matrix We then deduced the centralized kernel matrix $\tilde{\mathbf{K}}$. For each entry,

$$\begin{aligned}
\tilde{\mathbf{K}}_{ij} &= \phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j) \\
&= \phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j) - \frac{1}{N}\sum_{l=1}^{N}\phi(\mathbf{x}_i)^T\phi(\mathbf{x}_l) - \frac{1}{N}\sum_{l=1}^{N}\phi(\mathbf{x}_l)^T\phi(\mathbf{x}_j) \\
&- \frac{1}{N^2}\sum_{l=1}^{N}\sum_{k=1}^{N}\phi(\mathbf{x}_l)^T\phi(\mathbf{x}_k) \\
&= k(\mathbf{x}_i,\mathbf{x}_j) - \frac{1}{N}\sum_{l=1}^{N}k(\mathbf{x}_i,\mathbf{x}_l) - \frac{1}{N}\sum_{l=1}^{N}k(\mathbf{x}_l,\mathbf{x}_j) - \frac{1}{N^2}\sum_{l=1}^{N}\sum_{k=1}^{N}k(\mathbf{x}_l,\mathbf{x}_k)
\end{aligned}$$

Therefore,

$$\tilde{\mathbf{K}} = \mathbf{K} - 1_N\mathbf{K} - \mathbf{K}1_N + 1_N\mathbf{K}1_N$$

Hence, for the principal component of centralized featured data,

$$\tilde{\phi}(\mathbf{X})\mathbf{u}_i = \frac{1}{\sqrt{\lambda_i}}\tilde{\mathbf{K}}\mathbf{v}_i$$

where $\mathbf{v}_i$ is the eigenvector corresponding to the i-th large eigenvalue of $\tilde{\mathbf{K}}$.

Here, we finally compute the equation of kernel principal components. We apply the kernel PCA to the dataset with the oral bound we introduced before; the result is shown in the left of Figure 14.

The KPCA method divides the raw dataset into two groups, but for each group, the dataset merges into one point. This indicates that we may choose an inefficient kernel. Instead, we apply the KPCA with the most popular choice -radial basis function kernel- to the same dataset. The definition of the RBF kernel is

$$k(x_1,x_2) = \exp\left(-\frac{\|x_1-x_2\|^2}{2\sigma^2}\right)$$

where σ is a parameter.

The right of Figure 14 shows that the variance of Class 1 is larger than Class 0 in both kernel PC1 and kernel PC2 direction. According to different information one wants from the dataset, different kernel functions and feature functions can be chosen.

The references used for this section:

Bishop, C. M. (2006) Pattern recognition and machine learning. New York: Springer (Information science and statistics).[3]

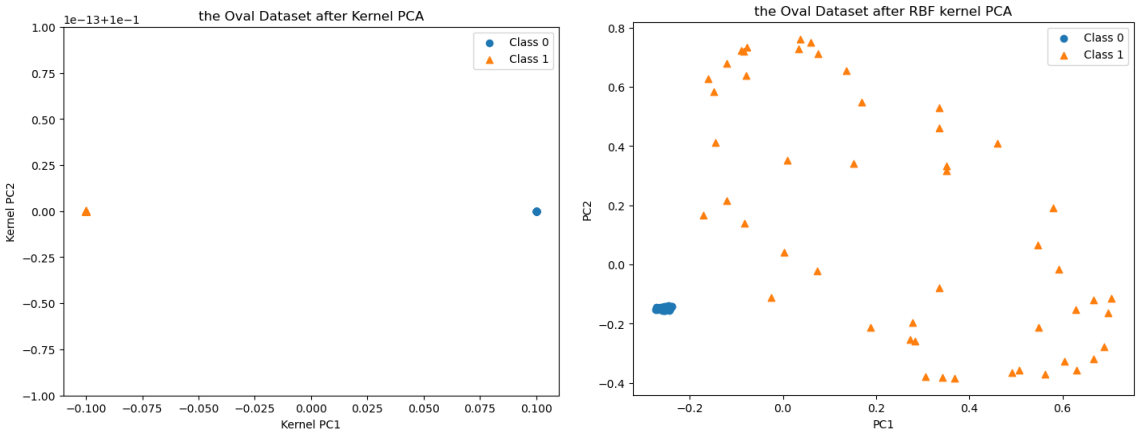


Figure 14: First 2 kernel Principal Components using Polynomial and the RBF kernels

5 Comparison of Different Methods

5.1 Principal Component Analysis and Linear Discriminant Analysis

In dimensionality reduction, Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA) emerge as two pivotal techniques, each with its distinct methodology and application spectrum. This section delves into their objectives, methodologies, and practical considerations to guide when to utilize one over the other.

Foundational Objectives and Methodological Differences PCA is fundamentally an unsupervised learning technique designed to identify the principal components that maximize variance within a dataset [5]. Its goal is to uncover underlying patterns by transforming the original data into a set of orthogonal features that represent the most significant directions of variance, thereby simplifying the data's complexity for further analysis [5]. For example, in real life, where a crowded room is filled with conversations, PCA helps identify the loudest voices that dominate the noise.

Conversely, LDA is a supervised learning method that utilizes class labels to enhance the separability between different classes. Its primary objective is to find the linear combination of features that best discriminates between classes, aiming for a projection of the data that maximizes class separation [11]. Using the previous analogy, if each group in the room belongs to different discussion topics, LDA would work to amplify the distinction between these topics, making it clear which group is discussing what.

Dimensionality Reduction Output PCA's output, the principal components, are orthogonal vectors that transform the data into a new subspace. This transformation is particularly beneficial for exploratory data analysis, revealing the data's intrinsic structure and facilitating visualization in lower-dimensional spaces [11, 13].

LDA produces discriminant functions based on linear combinations of input features that result in the most significant class separation. This output is instrumental in creating features that directly contribute to the effectiveness of classification algorithms, ensuring that the reduced-dimensional space is optimally aligned with the classification objectives [11, 13].

Interpretation and Performance PCA's principal components can be difficult to interpret since they are a combination of the original features and may not have direct, intuitive meanings. Despite this, PCA remains a powerful tool for data visualization and the initial exploration phase of data analysis projects [13].

By considering class labels in the dimensionality reduction process, LDA's discriminant functions clearly provide an advantage for classification tasks. These functions make LDA particularly adept at preparing data for classifiers, optimizing the separation between classes to improve classification accuracy [13].

Detailed Considerations for Application Both PCA and LDA operate under the assumption that data relationships are linear and that features interact through linear combinations. This

common ground forms the basis for their utility in reducing dimensionality while retaining meaningful information [1]. Feature scaling is essential for both methods, standardizing data features to have zero mean and unit variance, thereby ensuring equal contribution from all features in PCA and maintaining consistency across class covariance matrices in LDA [5, 11].

PCA assumes orthogonality among principal components, suggesting that perpendicular axes can effectively capture the data's structure, eliminating redundancy or correlation between new features. This characteristic underpins PCA's capacity to de-correlate and reduce dimensionality without significant information loss [5]. In statistical terms, this means that the covariance between any two components is zero [15]. However, PCA's reliance on variance to identify informative components assumes that higher variance equates to more valuable information. This assumption may not always hold true, as features with high variance may not necessarily be more relevant to specific analysis tasks, potentially leading to overlooked important patterns in less variable dimensions [13].

PCA is most effective in scenarios requiring a reduction of data dimensionality to uncover hidden structures or patterns, particularly in large datasets where the inherent dimensions make analysis computationally intensive or challenging [5]. It is also valuable in preprocessing steps to enhance the efficiency of complex machine learning models [13].

LDA is grounded in several key assumptions about the data it analyzes. It assumes that the data within each class follow a normal (Gaussian) distribution and for all classes to share the same covariance matrix, conditions known as normality and homoscedasticity, respectively [5]. These assumptions are crucial for LDA's effectiveness in maximizing class separability. Yet, deviations from these conditions, such as non-linear class distributions or unequal class covariances, can limit LDA's utility in complex datasets[8].

Dimensionality Reduction in the Iris Dataset The Iris dataset, with its distinct species characteristics, provides an ideal case study to explore the practical applications and effectiveness of PCA and LDA. Utilizing Python, the PCA and LDA techniques were applied to the Iris dataset, with the subsequent plots illustrating the outcomes:

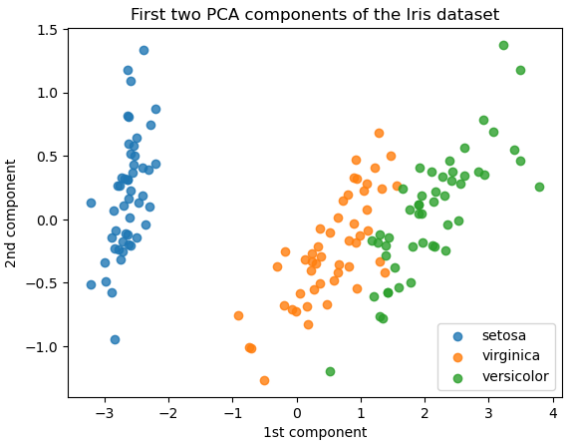


Figure 15: PCA Visualization of Iris Dataset

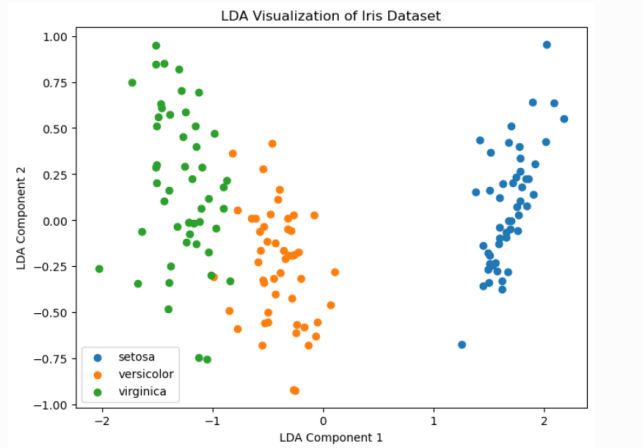


Figure 16: LDA Visualization of Iris Dataset

The PCA Plot 15 showcases the Iris dataset projected onto the first two principal components, which maximize the variance within the data:

- **Setosa Clustering:** The Setosa species is visibly distinct and separate from the other species, clustered tightly around the lower values on the first principal component. This separation highlights PCA's ability to recognize and emphasize the inherent variance within the Setosa species, effectively isolating it from Versicolor and Virginica.
- **Versicolor and Virginica Overlap:** The Versicolor and Virginica species are interspersed, demonstrating considerable overlap on the PCA plot. The absence of distinct clustering for these species suggests that PCA, while effectively capturing overall variance, does not necessarily address the subtle differences required to separate similar classes. This leads to a blended area where these two species merge, indicating a limitation of PCA in scenarios requiring precise class separation.

The LDA Plot 16, utilizing class labels, reveals the following:

- **Distinct Species Separation:** The LDA plot achieves a pronounced separation among all three species compared to the PCA plot. Setosa is well-separated, while Versicolor and Virginica are distinctly partitioned with minimal overlap. The clear delineation between these species demonstrates LDA's advantage when class labels are available, maximizing class separability.

- **Class-Specific Boundaries:** The plot reflects LDA's focus on optimizing the distance between different classes while minimizing the variance within each class. This targeted approach provides clear class boundaries and is especially conducive to classification and pattern recognition tasks.

The results of the PCA and LDA plots clearly show each technique's capabilities. PCA is a powerful tool for identifying broad trends and reducing dimensions without class label consideration, which can be particularly useful for unsupervised learning and initial data exploration. LDA, however, with its label-informed approach, excels in settings where the goal is to distinguish between predefined groups or classes, making it highly valuable for supervised learning and predictive modelling tasks. In applying these techniques to the Iris dataset, LDA more effectively facilitates classifying species, as the reduced-dimensionality space retains and emphasizes the critical information necessary for such differentiation. This comparison underscores the importance of selecting an appropriate dimensionality reduction method aligned with the specific goals and nature of the data.

5.2 Linear Principal Component Analysis and Kernel Principal Component Analysis

As mentioned in Section **??**, there are many kernel functions one can use for a sample dataset. Among all kernel functions, there is one type called polynomial kernel.

Definition 4. *Polynomial Kernel*

$$k(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j + c)^d$$

where $\mathbf{x}_i$ *and* $\mathbf{x}_j$ *are two samples.* d *is the degree of the polynomial and* c *is a constant term.*

Let's consider the case when degree $d = 1$ and $c = 0$. We then have,

$$k(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

Recall the definition of the kernel function; this indicates to $\phi(\mathbf{x}) = \mathbf{x}$, which is exactly the linear PCA method introduced in Section 3.2. Therefore, instead of comparing between methods, it's more like a comparison between linear and complex kernels.

Time Consuming As all the simple methods benefit from, the linear PCA methods compute faster and rely less on computer performance. This is due to easier code and less calculation. This can be extremely important and obvious when we have a huge dataset but only need a basic feature.

Recall the dataset with an oval boundary introduced in Section **??**.

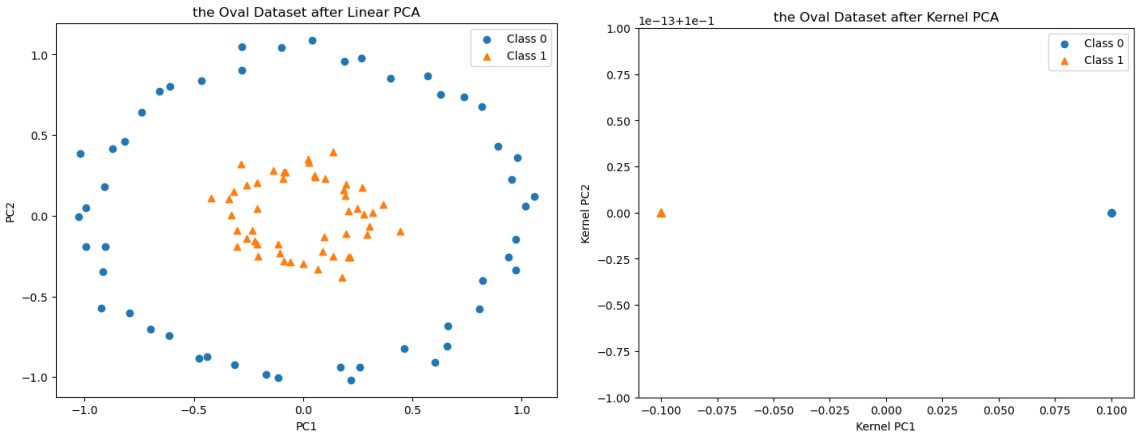


Figure 17: Comparison of Linear and Kernel PCA on the Oval data

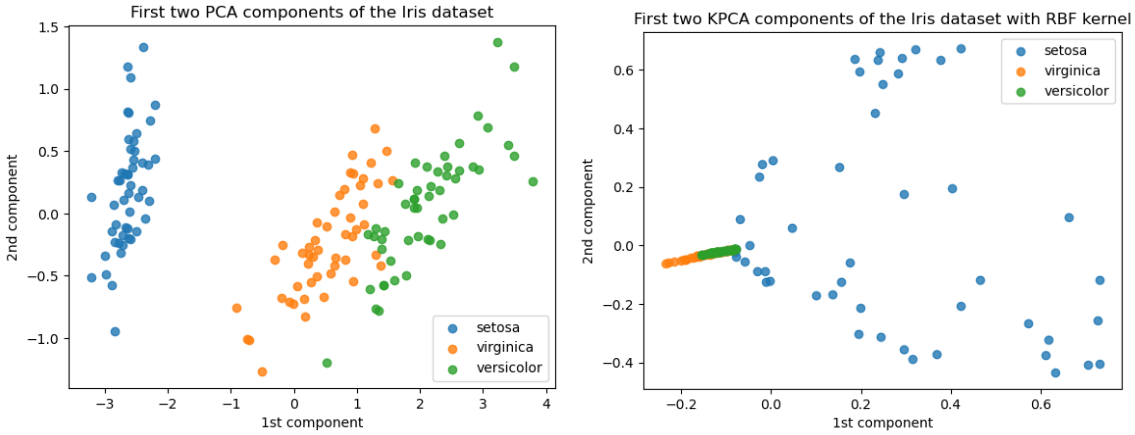


Figure 18: Comparison of Linear and Kernel PCA on the iris data

In the previous section, we showed that the linear PCA method reveals an overlap of Versicolor and Virginica. The RBF kernel PCA deduces that further and shows that these two groups have a linear correlation under the RBF kernel. While Linear PCA method already indicated the cluster of Setosa, Kernel PCA inhence the performance of cluster by increasing the variance.

The kernel method maps the original data to a feature space from the definition of feature functions and kernel functions. Therefore, if one has a specific space in which one would like to analyse the data, the KPCA method is better, and the Linear PCA method will not give a chance to achieve that.

Different Features Captured Another difference between linear kernels and complex kernels is that linear kernels concentrate on the original data. While complex kernels can capture the specific data feature one may want, such as polynomials in the oval dataset. This is also shown when applying these two methods to the iris dataset, where the KPCA method mapped the data to a Gaussian distribution space.

Unsupervised Although the Kernel PCA method increases the performance of the cluster, it's still not good at classification. As shown in Figure 18, there is an overlap between Versicolor and Virginica. Hence, Kernel PCA has many choices in kernel function, capturing different features of the raw data, but it is still an unsupervised method. One may tend to use the LDA method instead of Kernel PCA when it comes to classification.

6 Face Recognition (DR Application)

In recent years, many new methods have been created to help computers recognize faces. While advancements have been made in identifying faces with minor changes in expression or angle, developing methods for facial recognition when there are significant changes remains challenging [2]. Techniques for reducing dimensionality are effective in disregarding significant changes in facial expressions. These methods identify a linear projection that transitions faces from a space with high dimensionality to a feature space with low dimensionality, where facial expressions do not affect recognition [2].

Techniques like LDA, also known as FDA (Fisher Discriminant Analysis), and PCA are trained on a variety of facial images to pinpoint the most critical features distinguishing individuals based on their facial images [6]. To illustrate this concept, we reference a study by Benyamin Ghojogh et al. in their book [6] utilizing the AT&T facial dataset [7].

6.1 Projection Directions

The directions in which a facial dataset is projected can highlight facial features that are similar to ghost faces, which is why these projections are called *ghost faces* [6]. In the context of FDA and PCA, these ghost faces are known as *Fisherfaces* and *eigenfaces*, respectively. In Figure 19, the projection directions can isolate various facial features like the eyes, nose, lips, hair, and sunglasses [6]. The features extracted by FDA and PCA differ in their approach: PCA aims to maximize the

variance between faces, while FDA focuses on identifying that differentiate between distinct groups or classes of faces.



Figure 19: The projection direction of 3 FDA directions and the top 10 PCA directions for the first four classes of facial AT&T dataset. From *Elements of Dimensionality Reduction and Manifold Learning* [6]

6.2 Projection

The images projected onto FDA are shown in Figure 20, while the projections using PCA and kernel PCA are displayed in Figure 21.

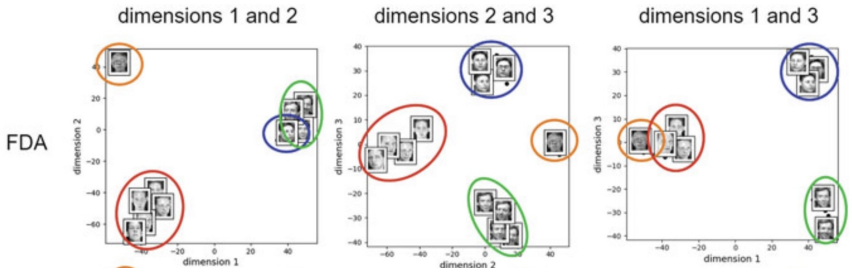


Figure 20: The projection of the first four classes of the AT&T dataset onto the FDA subspaces. From *Chapter 6.9 Elements of Dimensionality Reduction and Manifold Learning* [6]

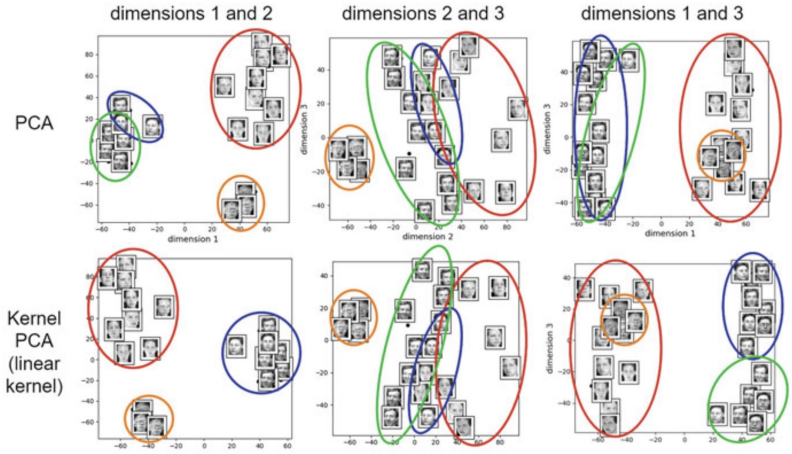


Figure 21: The projection of the first four classes of the AT&T dataset onto the PCA and kernel PCA subspaces. From *Chapter 5.7 Elements of Dimensionality Reduction and Manifold Learning* [6]

It is evident that FDA achieves better class separation compared to PCA and kernel PCA. This is because FDA uses class labels to distinguish between different classes in the subspace. On the other hand, PCA and kernel PCA focus solely on capturing the overall spread of the data (variance), without taking class labels into account, as discussed in Section 5.1).

6.3 Reconstruction

Figure 22 shows how some training images have been reconstructed. In this comparison, the PCA method provides better reconstruction overall than FDA. This is expected since PCA is specifically designed as a linear reconstruction method that minimizes the squared error. In contrast, as discussed in Section 5.1 the main objective of LDA, which FDA is a part of, is to separate different classes. Therefore, FDA focuses on maximizing the separation between classes, not on achieving high-quality reconstructed images.

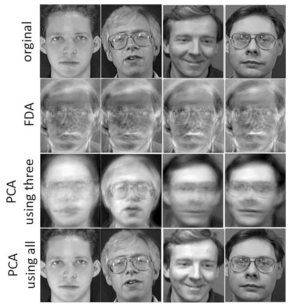


Figure 22: The reconstruction of four sample faces of AT&T datasets in FDA and PCA. From *Elements of Dimensionality Reduction and Manifold Learning* [6]

7 Conclusion

Dimensionality reduction, also referred to as feature extraction, is one of the important areas of machine learning and data science. In this report, we started with introducing the concept of data and the multiple forms it can take. These forms of raw data requires processing and transformation for effective analysis in data science. The discussion then extends to the challenges of big data; it highlights the critical role of data reduction methods in converting large datasets into meaningful information. Furthermore, we elaborate on the concept of datasets and delve into feature extraction. This process involves reducing the dimensionality of data without significant information loss. In the next section, it delves into the concept of dimensionality reduction (DR), particularly focusing on its importance in managing multidimensional data. It explains that such data consists of a collection of data points or feature vectors that are typically represented in matrix format. This representation facilitates the understanding of data tables where rows correspond to instances and columns to features. The process of DR is pertinent for big data with many correlated input variables. The reduction is achieved through a transformation that maps the original dataset to a new dataset with fewer dimensions. The section highlights that while DR is a critical step in data analysis, especially for big data, the choice of DR method can significantly impact the model's effectiveness. The suitability of a particular method vary depending on the task at hand and the specific characteristics of the data.

The next section delves into linear approaches to dimensionality reduction that aim to address the limitations posed by high-dimensional data, focusing on Spectral Dimensionality Reduction methods. Using the Iris dataset, the effectiveness of two geometric linear dimensionality reduction methods, PCA and LDA, is illustrated. PCA is highlighted as a technique that abstracts raw data into fewer components, maximizing the variance captured by principal components, which are linear combinations of the dataset's covariates. This process is shown to effectively reduce the iris dataset's dimensionality, with a notable reduction in complexity achieved by focusing on just two principal components. LDA is then introduced as a method that optimizes feature selection for class differentiation and it aims to maximize the ratio of between-class scatter to within class scatter,enhancing class separability. The application of LDA to the iris dataset demonstrates its ability to reduce dimensions while maintaining distinct class clusters, helping in classification tasks. The mathematical formulas and Python implementation provided for PCA and LDA underscore their roles in simplifying data, enhancing interpretability, and supporting more effective data analysis.

The section on KPCA, a non-linear dimensionality reduction method, explores how to handle data when the covariates do not exhibit linear relationships. KPCA extends PCA to non-linear data by introducing the concept of feature space. the transformation to this space allows the application of PCA in a manner where the linear relationships are preserved. This process is facilitated by

the kernel trick. Two examples are presented to illustrate the application of KPCA. In the first, a polynomial kernel fails to separate the classes effectively. In the second example, the RBF kernel is applied, showing better separation and indicating the variance between classes across the kernel principal components, which emphasizes the adaptability of KPCA to different data structures through the choice of kernel functions.

The choice between PCA, LDA, and KPCA depends on the nature of the data, the analysis goals, and the need for computational efficiency. While PCA and LDA offer foundational tools for understanding linear relationships and class distinctions, KPCA extends this framework to accommodate more complex data structures, offering a powerful tool for nuanced data exploration and analysis. Facial recognition is one of the most important applications of these methods, Each method inherent characteristics influence it effectiveness in various aspects of facial image analysis.

References

[1] Eugenia Anello. *What Is the Difference between PCA and LDA?* 2022. URL: https://365datascience.com/tutorials/python-tutorials/lda-vs-pca/.

[2] Peter N. Belhumeur, João P. Hespanha, and David J. Kriegman. "Eigenfaces vs. Fisherfaces: Recognition using class specific linear projection". In: *Lecture Notes in Computer Science* (2005).

[3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer New York, NY, 17 August 2006. ISBN: 978-0-387-31073-2.

[4] R. A. Fisher. *Iris*. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C56C76. 1988.

[5] Max Garzon et al. *Dimensionality Reduction in Data Science*. Springer Cham, 28 July 2022. ISBN: 978303105373. DOI: https://doi.org/10.1007/978-3-031-05371-9.

[6] Benyamin Ghojogh et al. *Elements of Dimensionality Reduction and Manifold Learning*. Springer Nature, 2023. ISBN: 9783031106026.

[7] The Digital Technology Group. *AT&T*. last accessed March 2024. URL: https://www.cl.cam.ac.uk/%20research/dtg/attarchive/facedatabase.html.

[8] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer New York, NY, 26 August 2009. ISBN: 978-0-387-84858-7. DOI: https://doi.org/10.1007/978-0-387-84858-7.

[9] I.T.Jolliffe. *Principle Component Analysis*. Springer, 2002.

[10] Richard Arnold Johnson and Dean W. Wichern. *Applied Multivariate Statistical Analysis*. Pearson Prentice Hall, 2007.

[11] Nirajan Khadka. *Ultimate Guide to Linear Discriminant Analysis (LDA)*. 2023. URL: https://dataaspirant.com/linear-discriminant-analysis/.

[12] Max Kuhn and Kjell Johnson. *Applied Predictive Modeling*. Springer New York, NY, 2013.

[13] Ajitesh Kumar. *PCA Vs LDA Differences, Plots, Examples*. 2022. URL: https://vitalflux.com/pca-vs-lda-differences-plots-examples/.

[14] Sylvain Lespinats, Benoit Colange, and Denys Dutykh. *Nonlinear Dimensionality Reduction Techniques*. Springer Cham, 2December 2021. ISBN: 978-3-030-81026-9. DOI: https://doi.org/10.1007/978-3-030-81026-9.

[15] The Pennsylvania State University - Eberly College of Science. *11.1 - Principal Component Analysis (PCA) Procedure*. n.d. URL: https://online.stat.psu.edu/stat505/lesson/11/11.1.